

Sprint #0 Report

Instructions

Objectives

- Make decisions on the Solitaire software development project.
- Learn unit testing and GUI programming in the language of your choice.

Deliverables and Grading Policy

Read the “CS 449 Homework Overview” document **carefully** and make the key decisions for the software development. Use the following template to complete your report.

1. Key Decisions of the Solitaire Project (2 points)

Topic	Decision
Object-oriented programming language (if not using OOP, then justify the use of the language)	Python
GUI library (strongly encouraged)	Tkinter
IDE (Integrated Development Environment)	VS Code
xUnit framework (e.g., JUnit for Java)	unittest
Programming style guide (must read it carefully)	Google Python Style https://google.github.io/styleguide/pyguide.html
Project hosting site	Github.com
Other decisions if applicable	

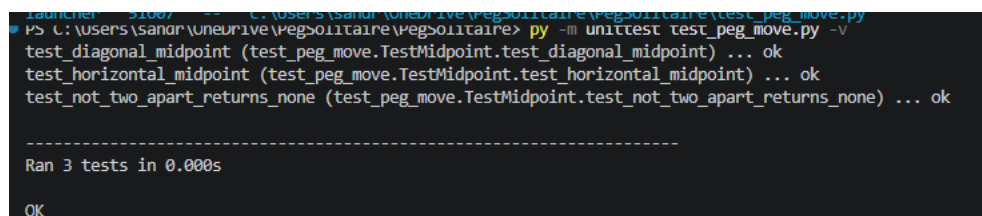
Sample programming style guides:

- Google Java Style Guide: <https://google.github.io/styleguide/javaguide.html>
- Google C++ Style Guide: <https://google.github.io/styleguide/cppguide.html>
- Google Python Style Guide: <https://google.github.io/styleguide/pyguide.html>

2. Unit testing (4 points)

Find a tutorial on the unit test framework you have chosen and write at least two xUnit tests of a program you have written or found elsewhere. Attach here (1) the screenshot of your program execution and (2) the source code of your program.

For example, Junit5 user guide: <https://docs.junit.org/5.10.3/user-guide/index.html>



```
launcher -- C:\Users\sandra\OneDrive\peg_solitaire\peg_solitaire\test_peg_move.py
PS C:\Users\sandra\OneDrive\peg_solitaire\peg_solitaire> py -m unittest test_peg_move.py -v
test_diagonal_midpoint (test_peg_move.TestMidpoint.test_diagonal_midpoint) ... ok
test_horizontal_midpoint (test_peg_move.TestMidpoint.test_horizontal_midpoint) ... ok
test_not_two_apart_returns_none (test_peg_move.TestMidpoint.test_not_two_apart_returns_none) ... ok

-----
Ran 3 tests in 0.000s

OK
```

```
import unittest
from peg_move import Position, midpoint, is_valid_move
```

```
class TestMidpoint(unittest.TestCase):
```

```

def test_horizontal_midpoint(self):
    start = Position(2, 0)
    end = Position(2, 2)
    self.assertEqual(midpoint(start, end), Position(2, 1))

def test_diagonal_midpoint(self):
    start = Position(0, 0)
    end = Position(2, 2)
    self.assertEqual(midpoint(start, end), Position(1, 1))

def test_not_two_apart_returns_none(self):
    start = Position(0, 0)
    end = Position(0, 1) # only one square away not a valid jump distance
    self.assertIsNone(midpoint(start, end))

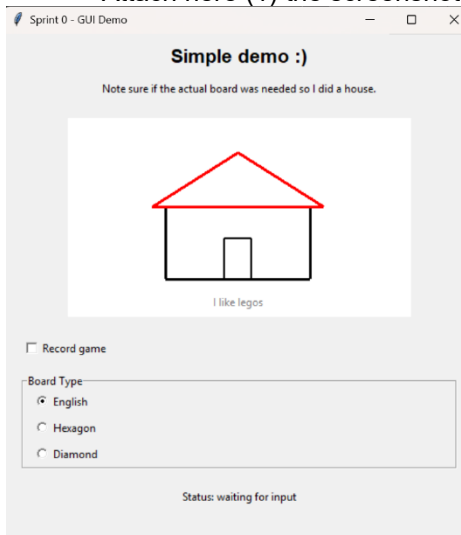
if __name__ == "__main__":
    unittest.main()

```

3. GUI programming (4 points)

Write a GUI program in the language you have chosen for your Solitaire project. The GUI of your program must include text, lines, a check box, and radio buttons. In this assignment, any GUI program of your own work is acceptable.

Attach here (1) the screenshot of your program execution and (2) the source code of your program.



```

import tkinter as tk
from tkinter import ttk

```

```

class DemoApp(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Sprint 0 - GUI Demo")
        self.geometry("420x360")

        #Text
        title_label = tk.Label(
            self, text="Simple demo :)", font=("Arial", 16, "bold")
        )
        title_label.pack(pady=10)

```

```

subtitle_label = tk.Label(
    self, text="Note sure if the actual board was needed so I did a house."
)
subtitle_label.pack(pady=(0, 10))

#Lines (drawn on a Canvas)
#A basic house outline: black rectangle walls and door
canvas = tk.Canvas(self, width=380, height=220, bg="white")
canvas.pack(pady=10)
self._draw_house(canvas, center_x=190, center_y=100)
canvas.create_text(190, 205, text="I like legos", fill="gray")

#Checkbox
self.record_game = tk.BooleanVar(value=False)
checkbox = tk.Checkbutton(
    self, text="Record game", variable=self.record_game,
    command=self.on_checkbox_toggle,
)
checkbox.pack(pady=(10, 5), anchor="w", padx=20)

#Radio buttons
board_frame = tk.LabelFrame(self, text="Board Type")
board_frame.pack(pady=10, padx=20, fill="x")

self.board_type = tk.StringVar(value="English")
for option in ("English", "Hexagon", "Diamond"):
    tk.Radiobutton(
        board_frame, text=option, variable=self.board_type,
        value=option, command=self.on_board_type_change,
    ).pack(anchor="w", padx=10, pady=2)

#Status label to show interaction feedback
self.status_label = tk.Label(self, text="Status: waiting for input")
self.status_label.pack(pady=10)

@staticmethod
def _draw_house(canvas, center_x, center_y):
    """Did a house outline on the canvas, centered at (center_x, center_y)."""

    #Walls (black rectangle)
    wall_left = center_x - 80
    wall_right = center_x + 80
    wall_top = center_y
    wall_bottom = center_y + 80

    canvas.create_line(wall_left, wall_top, wall_left, wall_bottom, fill="black", width=3)
    canvas.create_line(wall_right, wall_top, wall_right, wall_bottom, fill="black", width=3)
    canvas.create_line(wall_left, wall_bottom, wall_right, wall_bottom, fill="black", width=3)
    canvas.create_line(wall_left, wall_top, wall_right, wall_top, fill="black", width=3)

    #Roof (red triangle, overhanging the walls slightly)
    roof_left = (wall_left - 15, wall_top)
    roof_right = (wall_right + 15, wall_top)
    roof_peak = (center_x, wall_top - 60)

    canvas.create_line(*roof_left, *roof_peak, fill="red", width=3)
    canvas.create_line(*roof_peak, *roof_right, fill="red", width=3)
    canvas.create_line(*roof_left, *roof_right, fill="red", width=3)

    #Door (black rectangle, no top line so it reads as an opening)
    door_width, door_height = 30, 45

```

```
door_left = center_x - door_width // 2
door_right = center_x + door_width // 2
door_top = wall_bottom - door_height

canvas.create_line(door_left, wall_bottom, door_left, door_top, fill="black", width=2)
canvas.create_line(door_right, wall_bottom, door_right, door_top, fill="black", width=2)
canvas.create_line(door_left, door_top, door_right, door_top, fill="black", width=2)

def on_checkbox_toggle(self):
    state = "ON" if self.record_game.get() else "OFF"
    self.status_label.config(text=f"Status: Record game turned {state}")

def on_board_type_change(self):
    self.status_label.config(text=f"Status: Board type set to {self.board_type.get()}")

if __name__ == "__main__":
    app = DemoApp()
    app.mainloop()
```